评定成绩：_____

重庆邮电大学自动化学院

高级软件设计设计说明文档

设计题目： **“智能环境监测与管理系统”**

学 院 名 称 ：	自动化学院
组　　员 ：	姚宇杰，庹新宇，陈诗琪，邵彦淇，张馨怡，罗雯丽，刘江鹏，李双江
专　　业 ：	自动化
指 导 教 师 ：	岑明

填表时间：2025 年 3 月

重庆邮电大学

一、引言

1.1 编写目的

随着物联网技术的发展，对环境参数（如温湿度、光照等）进行智能、远程、集中的监测与调控需求日益增长，尤其在智能家居、智慧农业、仓储管理等领域。传统单点、本地化的传感器系统功能受限。本项目旨在设计并实现一个基于STM32的多模态温湿度传感器监控系统，该系统不仅能实现前端环境数据的实时采集与本地控制，还能通过通信模块将数据上传至后台服务器进行存储与分析，并由上位机软件进行远程监控与管理，从而构建一个更完整、更智能的环境监测解决方案。

1.2 读者对象和阅读建议

本报告主要面向环境监测、嵌入式系统开发、物联网应用、软件工程等领域的工程师、科研人员、技术管理人员及相关专业学生。建议读者首先理解系统的整体架构和核心功能，然后根据兴趣深入阅读前端嵌入式设计、通信协议、后台服务及上位机应用等具体模块的实现细节。

1.3 项目范围

本项目范围主要包括：

前端数据采集与控制节点：

1. 基于STM32F103C8T6微控制器的主控系统设计。
2. 温湿度传感器和光敏传感器的数据采集与处理。
3. 本地OLED显示、声光报警及风扇等设备联动控制。
4. 通过按键进行本地参数设置。

数据通信模块：

1. 实现前端节点与后台服务器之间的数据传输（例如，使用Wi-Fi模块或有线以太网）。

二、总体描述

2.1 总体设计

本系统设计为一个三层架构：感知执行层、网络传输层和应用服务层。

感知执行层： 以 STM32F103C8T6 单片机为核心，连接温湿度传感器、光敏传感器进行环境数据采集。该层还包括 OLED 显示、LED 报警灯、蜂鸣器、按键以及风扇驱动等模块，实现本地数据显示、报警和控制。

网络传输层： STM32 通过外接通信模块（如 ESP8266 Wi-Fi 模块或 W5500 以太网模块）将采集到的数据按照预定协议（如 MQTT 或 HTTP）打包，并通过无线或有线网络发送到后台服务器。同时，也能接收来自后台服务器的指令。

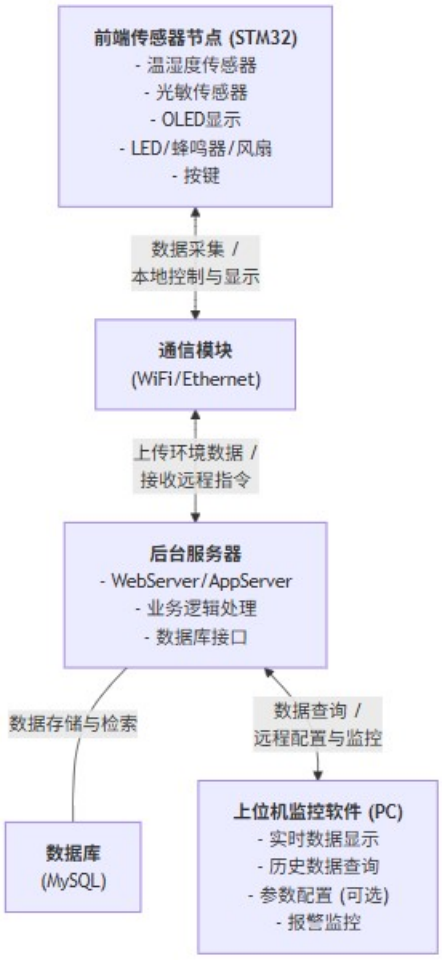
应用服务层： 包括后台服务器和上位机监控软件。

后台服务器： 部署在云端或本地服务器，负责接收、解析并存储来自前端节点的数据到数据库（如 MySQL）。服务器端也可能包含 Web 服务接口，供上位机或其他客户端访问数据。

上位机监控软件： 运行在 PC 端，通过网络从后台服务器获取数据，实现数据显示、历史查询、参数配置、报警监控等功能。

2.2 系统架构

前端传感器节点（STM32）包含温湿度传感器、光敏传感器、OLED 显示、LED/蜂鸣器/风扇和按键，负责数据采集、本地控制与显示。数据通过通信模块（WiFi/Ethernet）上传环境数据并接收远程指令。后台服务器（WebServer/AppServer）处理业务逻辑，提供数据库接口，与数据库（MySQL）进行数据存储与检索。上位机监控软件（PC）实现实时数据显示、历史数据查询、参数配置（可选）和报警监控，通过数据查询和远程配置与监控与后台服务器交互。



2.3 运行环境

前端节点：硬件：STM32F103C8T6 微控制器及外围传感器、执行器。软件：嵌入式 C 语言，Keil MDK 等开发环境。

后台服务器：操作系统 Windows Server。

Web 服务器：Nginx 。

应用服务器/语言：Node.js, Python (Flask/Django), Java (Spring Boot)等。

数据库：MySQL 5.7+

上位机软件：

操作系统：Windows 11

开发语言/平台：**上位机程序可选用 C++**

通信协议：

MQTT，HTTP/HTTPS，TCP/IP，CoAP 等。

2.4 设计和实现上的约束

前端 STM32 程序采用 C 语言编写。

上位机程序可选用 C++ QT。

数据库设计应满足数据存储和查询效率的需求。

系统应具备一定的可扩展性，方便未来增加新的传感器或功能。

三、系统功能实现

3.1 前端节点功能

环境参数采集： 实时采集环境温度、湿度、光照强度。

本地数据显示： 在 OLED 屏上显示当前温湿度、光照状态、设定阈值。

本地报警与控制：

温度超限：LED 闪烁报警，启动风扇。

湿度超限：蜂鸣器鸣响报警。

光照自动控制：根据昼夜状态控制照明 LED。

本地参数设置： 通过按键调整温湿度报警阈值。

数据上传： 将采集的环境数据和设备状态通过通信模块发送至后台服务器。

指令接收与执行（可选）： 接收后台服务器下发的控制指令（如远程修改阈值、控制风扇）。

3.2 后台服务器功能

数据接收与解析： 接收来自前端节点的数据包，并按协议解析。

数据校验与处理： 对数据进行有效性校验和必要的转换。

数据存储： 将处理后的环境数据、报警记录、设备状态等存入数据库。

数据服务接口： 提供 API 接口（如 RESTful API）供上位机或其他客户端请求数据（实时数据、历史数据）。

设备管理（可选）： 记录在线设备状态，管理设备信息。

指令下发（可选）： 向指定前端节点发送控制指令。

3.3 上位机监控软件功能

实时数据显示：

以数字、仪表盘或曲线图等形式实时展示从后台获取的各监 测点的温湿度、光照等数据。

历史数据查询与展示：

按时间范围查询指定监测点的历史数据。

以表格和曲线图的形式展示历史数据，分析变化趋势。

报警监控与记录：

当接收到后台转发的报警信息或监测数据超限时，在界面上进行醒目提示（如声音、颜色变化）。

记录报警事件的时间、类型、数值等信息。

4.1 前端嵌入式软件设计

4.1.1 软件总体流程

系统上电后，进行初始化（GPIO、通信模块、传感器、OLED）。主循环中周期性执行：

采集温湿度、光照数据。

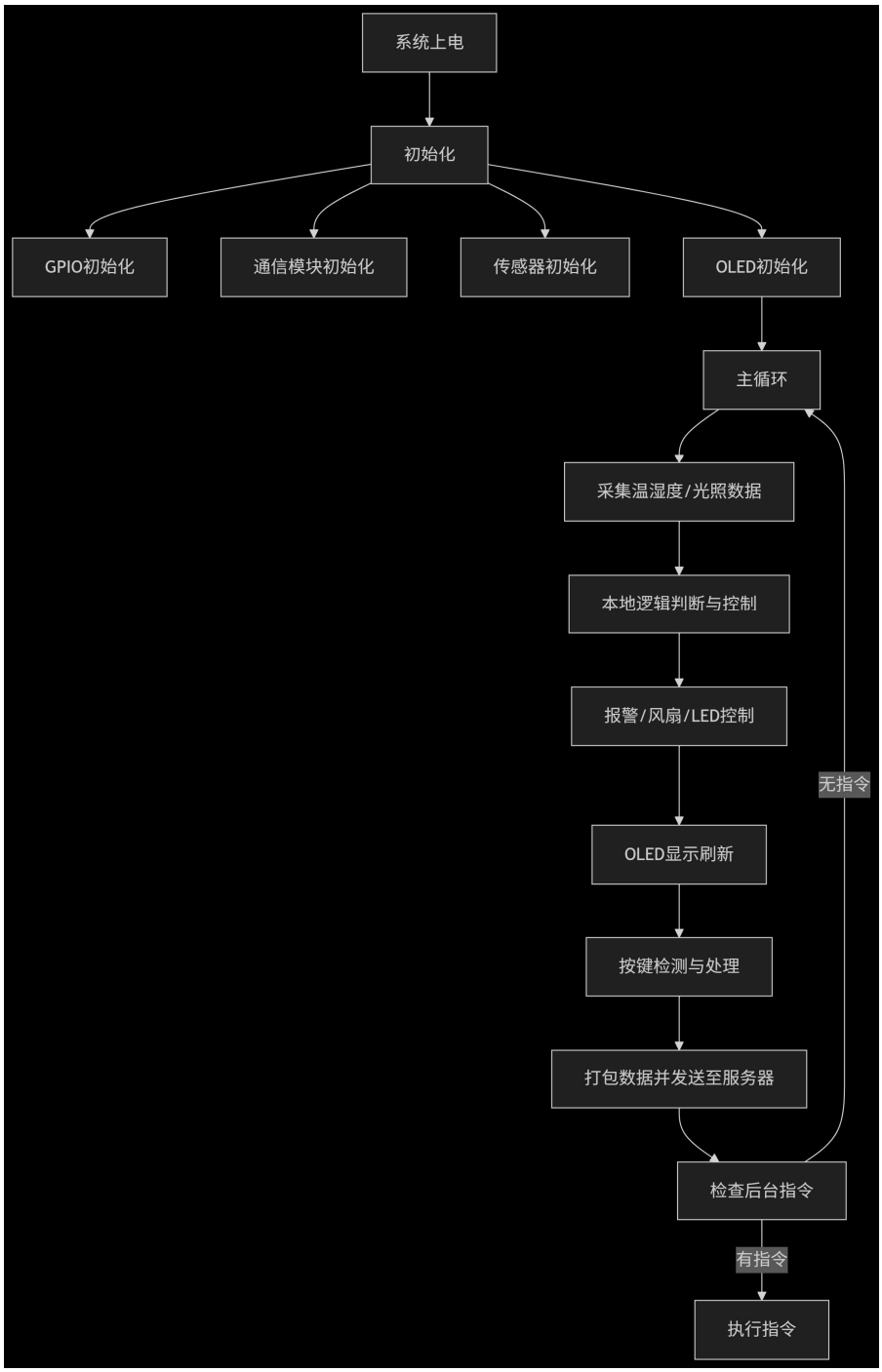
本地逻辑判断与控制（报警、风扇、照明 LED）。

OLED 显示刷新。

按键检测与处理。

打包数据并通过通信模块发送至后台服务器。

检查是否收到后台指令，并执行



4.1.2 主要功能模块

传感器驱动模块、OLED 显示模块、按键处理模块、报警与控制模块（基本同前）。

通信模块驱动： 封装函数用于初始化通信模块、连接网络/服务器、发送数据、接收数据。

4.2 通信设计

4.2.1 通信方式选择

前端与后台之间可选用 Wi-Fi（如 ESP8266/ESP32）或 有线以太网（如 W5500 模块）。

上位机与后台服务器之间通常使用 TCP/IP 协议（如 HTTP 或自定义 Socket）。

4.2.2 通信协议选择

前端 <-> 后台：

HTTP： 前端定期向后台 API 发送 POST 请求上报数据。相对 MQTT 开销稍大。

上位机 <-> 后台：

HTTP： 上位机通过 GET 请求获取数据，通过 POST/PUT 请求修改 配置（如果支持）。

4.2.3 数据格式

前端上报数据示例（JSON）：

```json
{
  "deviceId": "STM32_NODE_001",
  "timestamp": 1678886400,
  "temperature": 25.5,
  "humidity": 60.2,
  "light_level": 800, // 假设是光敏电阻原始值或勒克斯值
  "alarm_temp": false,
  "alarm_humidity": true}
```

后台下发指令示例（JSON, for MQTT）： 主题： control/STM32_NODE_001/set_threshold 负载：

```json
{
  "temp_threshold_high": 30.0,
  "humidity_threshold_high": 70.0}
```

4.3 后台服务器软件设计

4.3.1 技术选型

语言/框架：, Python (Flask/Django)

数据库：MySQL,

4.3.2 主要模块

设备接入模块： 负责处理前端节点的连接请求（HTTP 请求 监听）。

数据解析与校验模块： 解析传入的数据包，校验数据合法性。

业务逻辑处理模块：

根据数据内容判断是否触发报警，并可能需要通知上位机或执行其他动作。

处理上位机的配置请求，并可能需要下发指令给前端。

数据存储模块： 调用数据库接口，将数据写入数据库。

API 接口模块： 提供 RESTful API 或其他形式的接口供上位机查询数据。

4.4 数据库设计

4.4.1 主要数据表设计

设备表 (devices)

device_id VARCHAR(50) PRIMARY KEY (设备唯一标识)

device_name VARCHAR(100) (设备名称/位置)

status VARCHAR(20) (在线/离线)

last_seen TIMESTAMP (最后通讯时间)

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

传感器数据表 (sensor_data)

id BIGINT AUTO_INCREMENT PRIMARY KEY

device_id VARCHAR(50) (外键关联 devices 表)

timestamp TIMESTAMP (数据采集时间)

temperature DECIMAL(5,2) (温度)

humidity DECIMAL(5,2) (湿度)

light_level INT (光照强度)

INDEX idx_device_timestamp (device_id, timestamp)

报警记录表 (alarm_logs)

id BIGINT AUTO_INCREMENT PRIMARY KEY

device_id VARCHAR(50) (外键关联 devices 表)

alarm_type VARCHAR(50) ('TEMPERATURE_HIGH', 'HUMIDITY_HIGH', etc.)

alarm_value DECIMAL(10,2) (触发报警时的数值)

threshold_value DECIMAL(10,2) (当时的阈值)

timestamp TIMESTAMP (报警发生时间)

is_cleared BOOLEAN DEFAULT FALSE (报警是否已解除)

cleared_at TIMESTAMP NULL (报警解除时间)

设备配置表 (device_configs) (可选)

device_id VARCHAR(50) PRIMARY KEY (外键关联 devices 表)

temp_threshold_high DECIMAL(5,2)

humidity_threshold_high DECIMAL(5,2)

config_updated_at TIMESTAMP

4.5 上位机软件设计

4.5.1 技术选型

C++ QT 开发上位机。

4.5.2 主要界面与功能模块

主界面：

设备列表/选择区域。

实时数据显示区（仪表盘、数值、简易曲线）。

报警状态指示区。

功能导航（跳转到历史数据、参数设置等）。

实时数据显示模块： 定期从后台 API 拉取最新数据或通过 WebSocket/MQTT 接收推送，并更新界面。

历史数据查询模块：

提供日期时间选择控件。

向后台 API 发送查询请求。

将返回的数据以表格和折线图形式展示。

参数设置模块 (可选)：

显示当前设备的阈值等参数。

允许用户修改并提交到后台 API。

报警处理模块：

接收后台的报警信息，进行弹窗、声音或颜色提示。

显示报警历史列表。

网络通信模块：

封装 HTTP 客户端，负责与后台服务器交互。

5. 测试用例

本章将描述“智能环境监测与管理系统”的测试用例，主要验证模拟前端、后台服务器和简化版上位机之间的核心功能和交互。

5.1 测试策略

本系统将采用以下测试策略：

功能测试：验证模拟前端、后台服务器和上位机各自的核心功能是否按预期工作。

集成测试：验证模拟前端与后台、上位机与后台之间的数据传输和操作是否正确。

5.2 测试环境

模拟前端节点：运行 Python 模拟程序的计算机环境。

后台服务器：部署有 Nginx、应用服务（Python/Node.js/Java）和 MySQL 5.7+ 数据库的服务器环境。

上位机：运行 C++ QT 应用程序的 Windows 11 计算机。

网络环境：确保模拟前端、后台服务器和上位机之间网络连通。

5.3 测试用例详情

以下列举各模块的关键测试用例。

5.3.1 模拟前端节点（Python）功能测试（查看 附录 4 的结果 1 2 3）

用例 ID	测试目标	前置条件	测试步骤	预期结果
FND_SIM_001	验证模拟数据生成	Python 脚本已准备好。	运行 Python 模拟前端脚本。	脚本能周期性地生成模拟的温湿度、光照等环境数据。

用例 ID	测试目标	前置条件	测试步骤	预期结果
FND_SIM_002	验证模拟报警逻辑	Python 脚本已准备好。	1. 修改脚本中的模拟数据，使其超过预设阈值。 2. 观察脚本输出或内部状态。	脚本能根据模拟数据是否超限，输出或记录相应的模拟报警状态（例如，打印“温度超限”）。
FND_SIM_003	验证数据发送到后台	Python 脚本已准备好，后台服务器运行中。	运行 Python 模拟前端脚本，观察脚本输出。	脚本能成功将模拟数据（包括报警状态）发送到后台服务器，无连接错误。

5.3.2 后台服务器功能测试（查看 附录 4 的结果 4 5）

用例 ID	测试目标	前置条件	测试步骤	预期结果
BKD_001	验证数据接收与存储	后台服务器运行中，数据库连接正常。	运行模拟前端 Python 脚本，使其发送数据到后台。	后台服务器能成功接收模拟前端发送的数据，并将其正确存储到 MySQL 数据库的 sensor_data 表中。
BKD_002	验证 API 接口可用性	后台服务器运行中。	使用 API 测试工具（如 Postman)尝试访问后台提供的 API 接口（例如，用于上位机连接、添加/删除数据的接口）。	API 接口能正常响应，返回正确的 HTTP 状态码。

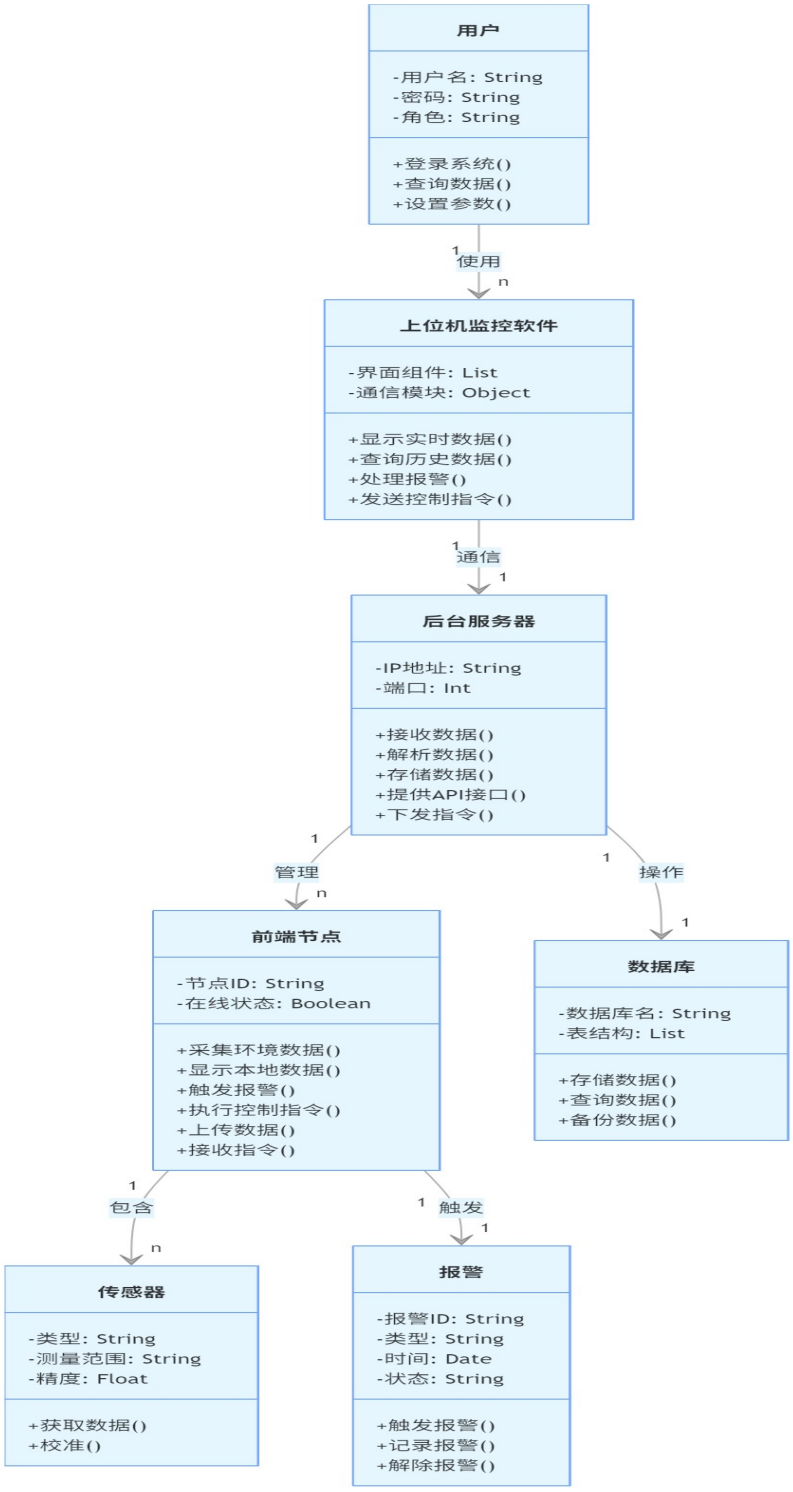
5.3.3 上位机监控软件功能测试（查看 附录 4 的结果 7 8 9）

用例 ID	测试目标	前置条件	测试步骤	预期结果
CLI_001	验证连接成功显示	上位机程序已启动，后台服务器运行中。	1. 启动上位机程序。 2. 尝试连接后台服务器。	上位机界面能显示“连接成功”状态或提示。
CLI_002	验证添加数据功能	上位机已成功连接后台，数据库界面已打开。	1. 在上位机“数据库界面”中，输入要添加的数据（例如，新的设备 ID 或配置参数）。	1. 上位机提示“添加成功”。 2. 检查后台 MySQL 数据库，新数据已

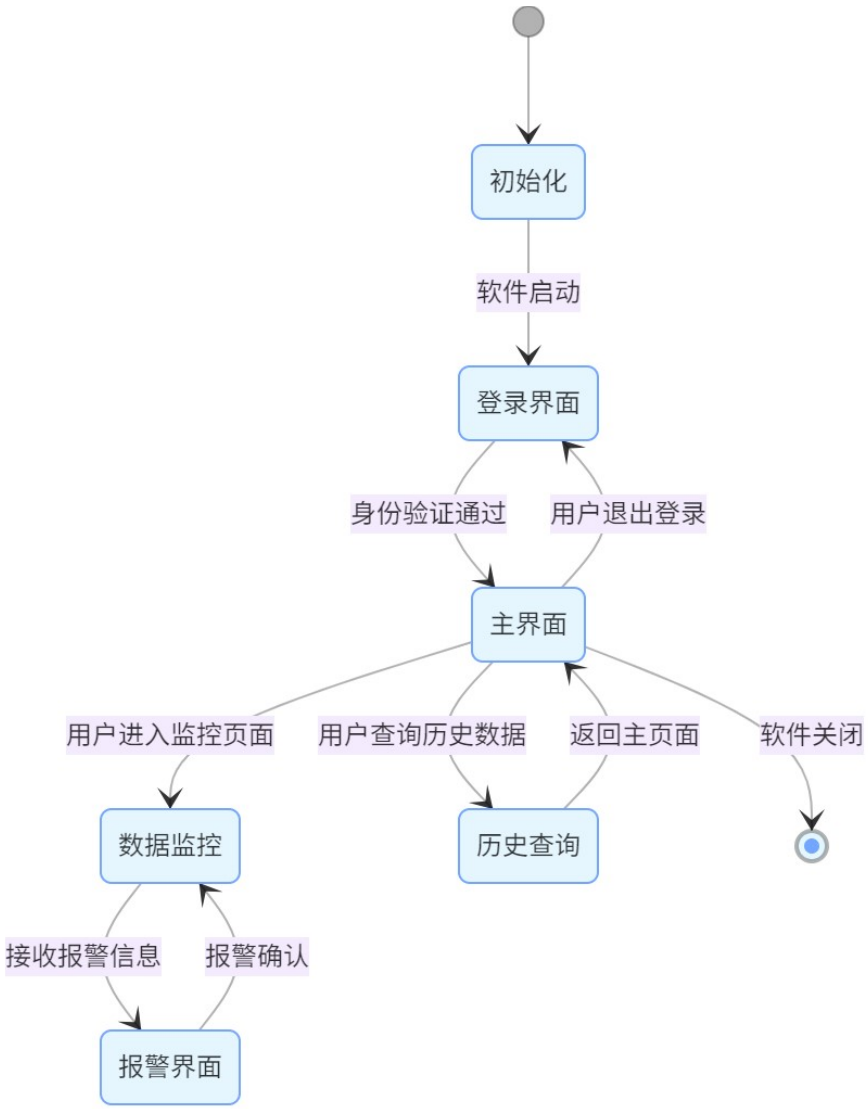
用例 ID	测试目标	前置条件	测试步骤	预期结果
			2. 点击“添加数据”按钮。	成功插入到对应表中。
CLI_003	验证删除数据功能	上位机已成功连接后台，数据库界面已打开，数据库中有可删除的数据。	1. 在上位机“数据库界面”中，选择或输入要删除的数据的标识（例如，设备 ID）。 2. 点击“删除数据”按钮。	1. 上位机提示“删除成功”。 2. 检查后台 MySQL 数据库，对应数据已成功从表中删除。
CLI_004	验证连接失败处理	上位机程序已启动，后台服务器停止运行或网络断开。	1. 启动上位机程序。 2. 尝试连接后台服务器。	上位机界面能显示“连接失败”或“网络错误”等提示信息。

附录

1. 类图：

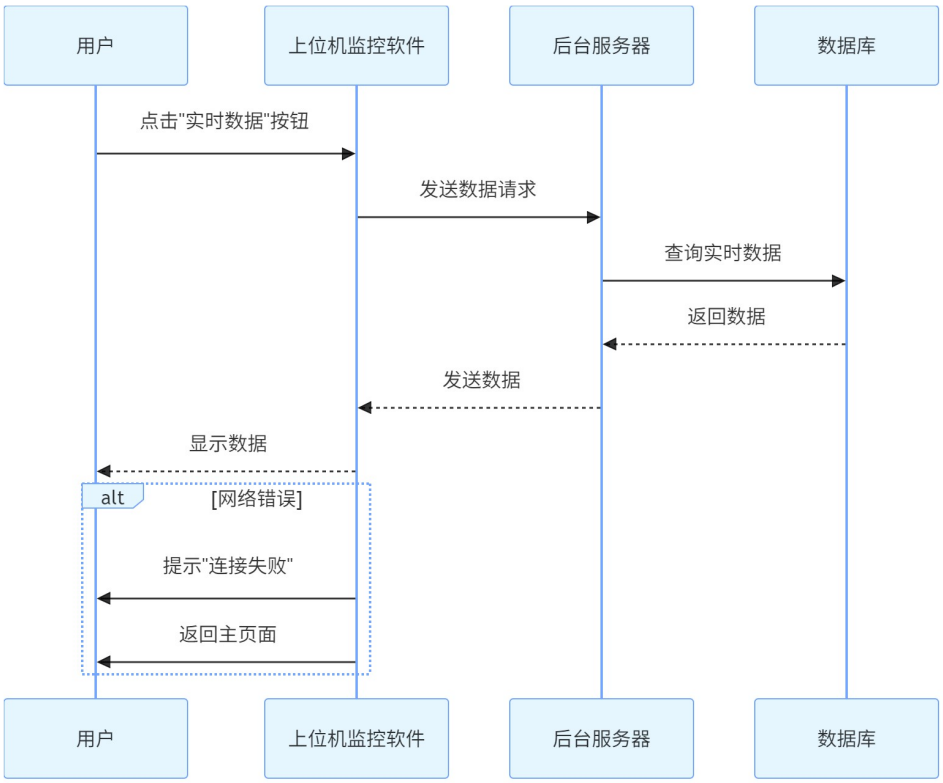


2. 上位机状态图：

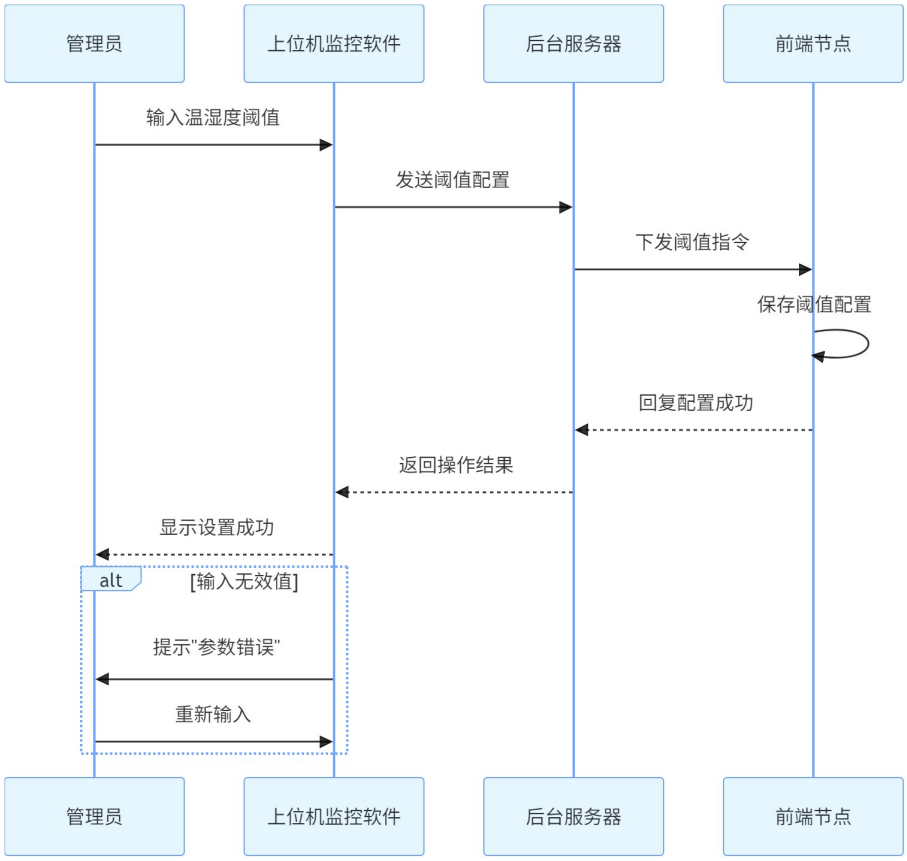


3. 顺序图：

3．1 查询数据顺序图：

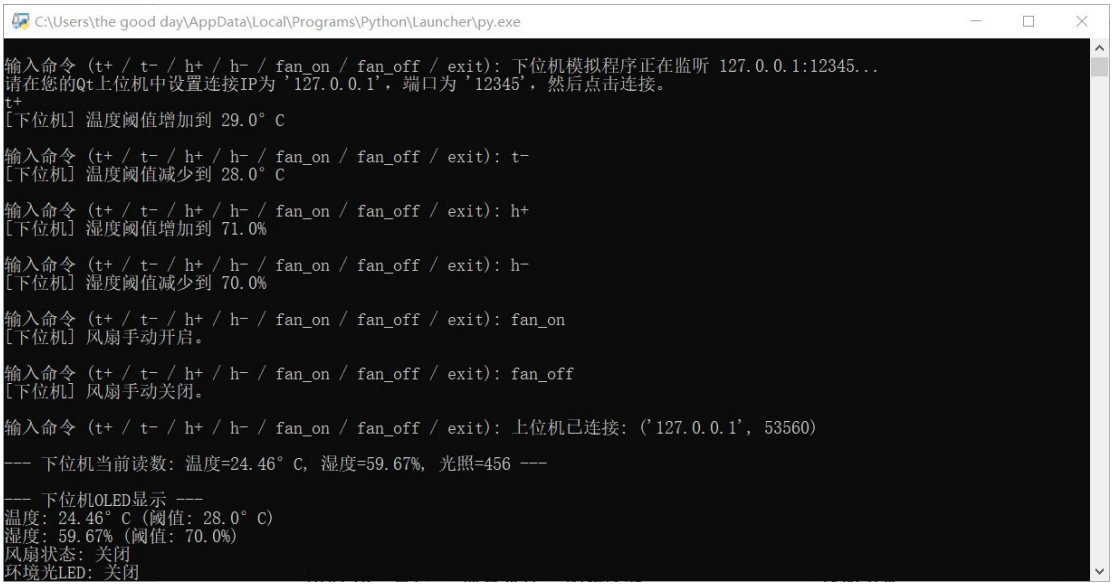

用户
上位机监控软件
后台服务器
数据库
点击"实时数据"按钮
发送数据请求
查询实时数据
返回数据
发送数据
显示数据
alt
[网络错误]
提示"连接失败"
返回主页面
用户
上位机监控软件
后台服务器
数据库

3.2 设置报警阈值用例顺序图：

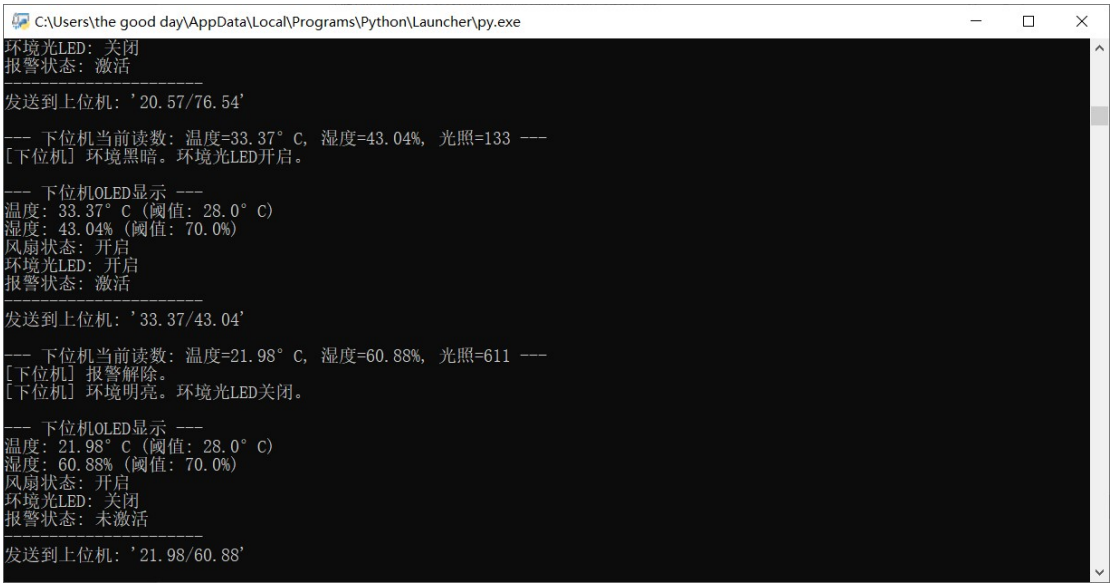


4. 测试用例结果

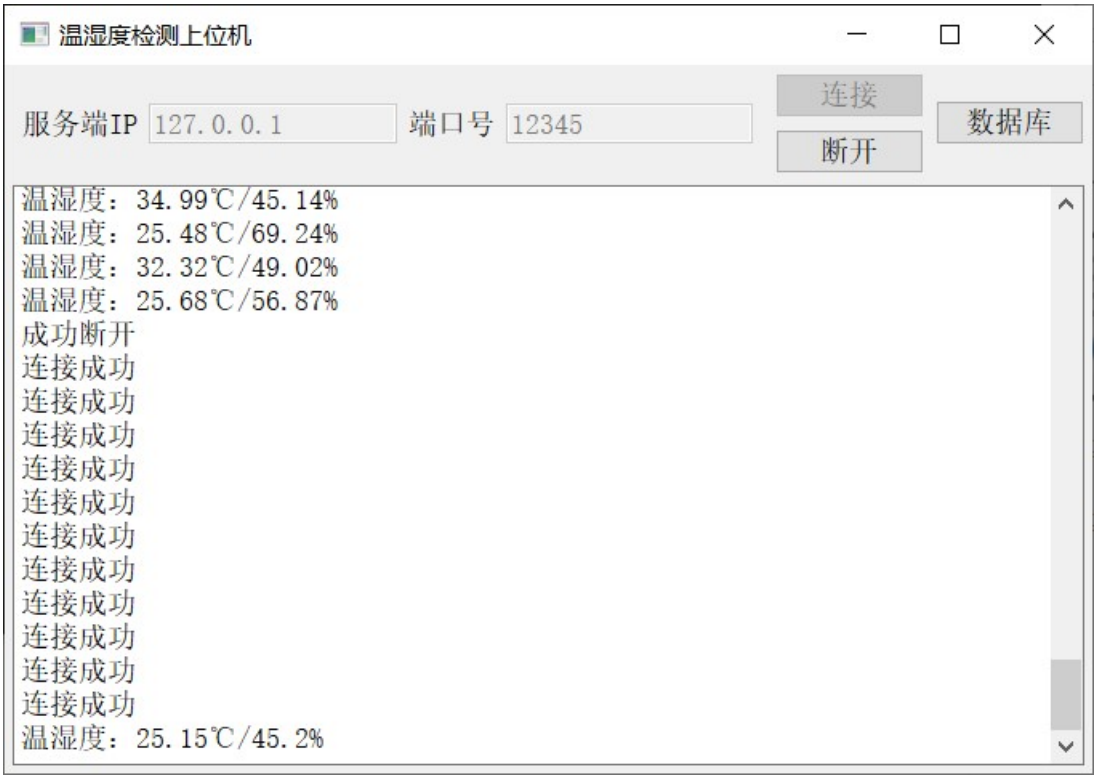
4.1 前端节点测试结果 1



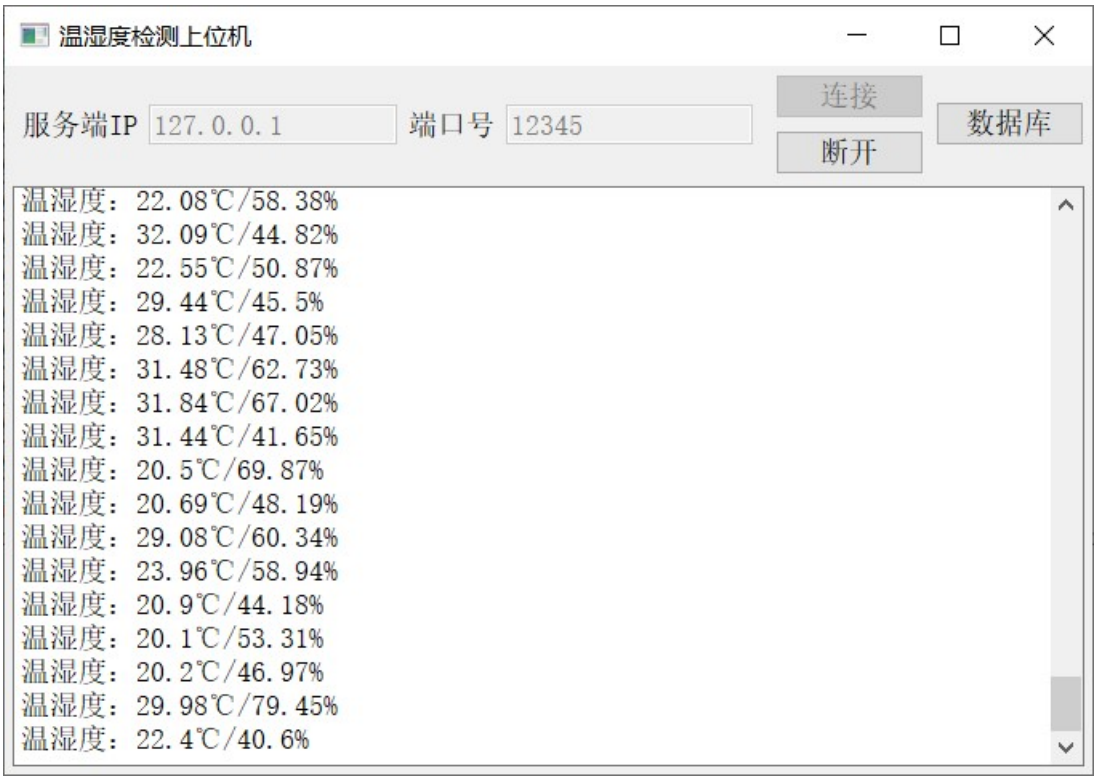
4.2 前端节点测试结果 2



4.3 上位机前端测试结果 1

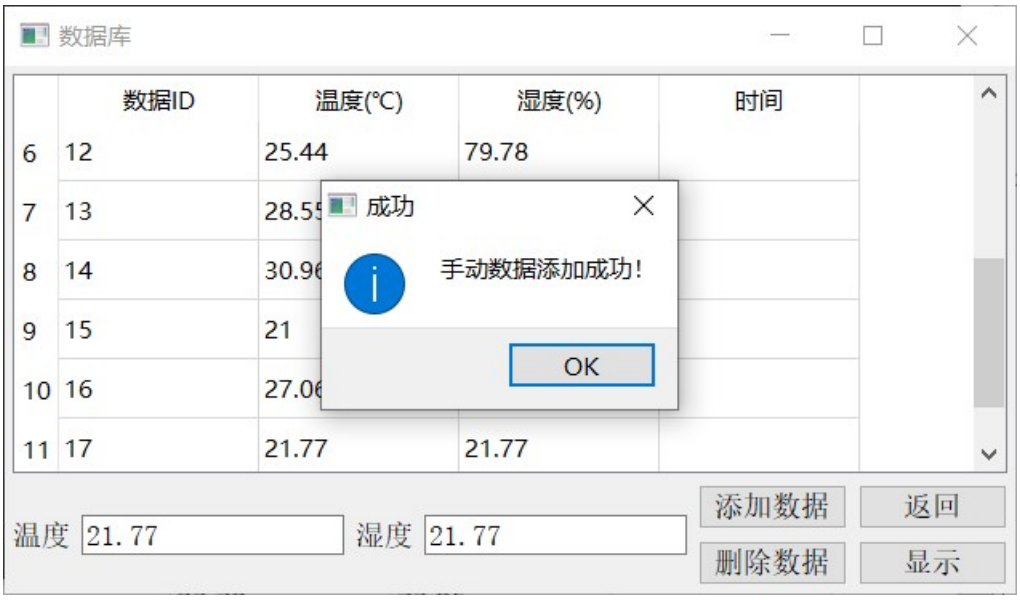


4.4 上位机前端测试结果 2

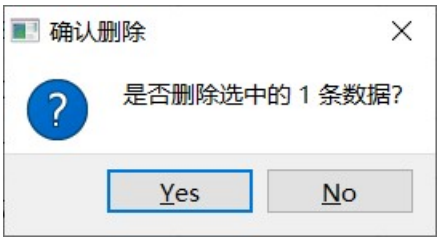


4.5 数据库测试结果 1

4.6 数据库测试结果 2


数据库
数据ID
温度(℃)
湿度(%)
时间
6 12 25.44 79.78
7 13 28.55
8 14 30.96
9 15 21
10 16 27.06
11 17 21.77 21.77
成功
手动数据添加成功！
OK
温度 21.77
湿度 21.77
添加数据
返回
删除数据
显示

4.7 数据库测试结果 2


确认删除
是否删除选中的 1 条数据?
Yes
No